CV Model → LLM Pipeline
Technical Handoff Document

Project	Decoupled Microservices Framework for Spatio-Temporal Detection of Pre-Attentive Cognitive Fatigue
Prepared by	CV Model Team
Intended for	LLM / XAI Integration Team
Date	13 September 2026
Version	1.0

This document describes the output interface of the CV fatigue detection model and provides everything the LLM/XAI team needs to begin building the explanation and alerting layer. No prior knowledge of the CV training process is required.

Contents

1 Overview **2**

2 Class Taxonomy **2**

3 Dataset & Training Pipeline **2**
- 3.1 Source Datasets . . . 2
- 3.2 Final Merged Dataset Sizes . . . 3
- 3.3 Preprocessing Pipeline . . . 3

4 Model Output Specification **4**
- 4.1 ONNX Model: `efficientnet.onnx` . . . 4
- 4.2 Running Inference . . . 4
- 4.3 Per-Frame JSON Payload . . . 5

5 Sliding Window & Temporal Context **6**

6 Suggested LLM Input Schema (MCP Payload) **6**

7 Suggested LLM System Prompt Template **7**

8 Severity Threshold Guidelines **8**

9 Files Delivered by the CV Team **8**

10 Open Questions for Team Alignment **8**

1. Overview

The computer vision (CV) model analyses individual face image crops extracted from a live video stream and outputs a **probability distribution across three fatigue states**. The LLM/XAI layer consumes these per-frame predictions, reasons over a sliding temporal window, and produces a human-readable explanation of the driver's current cognitive state along with a severity rating and recommended action.

Figure 1 summarises the end-to-end data flow.

```
Face crop (JPEG)
↓   Preprocessing (resize, normalise)
↓   EfficientNet-B0 ONNX
↓   Softmax ⇒ 3-class probabilities
↓   FastAPI JSON payload
↓   MCP Server (sliding window buffer)
↓   Local LLM (Ollama)
↓   Structured JSON alert ⇒ React UI
```

2. Class Taxonomy

The model classifies each face crop into one of **three states**. The binary fatigue/alert decision is derived from these three classes as shown below.

Class ID	Class Name	Description	Fatigue Mapping
0	**microsleep**	Eyes visibly closed or nearly shut; involuntary pre-sleep episode; head may droop.	**FATIGUE**
1	**neutral**	Eyes fully open, face alert and attentive; normal driving state.	**ALERT**
2	**yawning**	Mouth wide open in a yawn; eyes may be partially closed; early fatigue indicator.	**FATIGUE**

Table 1: Three-class fatigue taxonomy used by the CV model.

NOTE **Why 3 classes instead of binary?** A binary alert/fatigue signal tells you *that* something is wrong, not *what*. Distinguishing microsleep from yawning lets the LLM produce specific, actionable explanations — "the driver's eyes have been closing repeatedly" is more useful than "the driver appears fatigued."

3. Dataset & Training Pipeline

3.1 Source Datasets

Three datasets were combined to train the model:

Label unification for Datasets A & B:

- `active` / `NATURAL` → class `1` (neutral)

Dataset	Images	Type	Original Labels
`images/` (A)	12,102	48×48 grayscale crops	active / fatigue
`Drowsy_datset/` (B)	10,302	48×48 grayscale crops	NATURAL / DROWSY
`train/valid/test/` (C)	14,297	Full-frame colour + YOLO bboxes	microsleep / neutral / yawning

Table 2: Source datasets and their original label schemes.

- `fatigue` / `DROWSY` → class `0` (microsleep)

Dataset C provides native 3-class labels and bounding boxes that locate the face within each full-frame image.

3.2 Final Merged Dataset Sizes

Split	Total	microsleep (0)	neutral (1)	yawning (2)
Train	26,854	9,377 (34.9%)	13,544 (50.4%)	3,933 (14.6%)
Val	4,783	1,830 (38.3%)	2,581 (54.0%)	372 (7.8%)
Test	5,040	2,058 (40.8%)	2,798 (55.5%)	184 (3.6%)

Table 3: Final merged dataset split sizes and class distributions.

CAUTION **Class imbalance warning:** The `yawning` class is underrepresented (≈14% of training data). The model uses class-balanced loss weighting and MixUp augmentation to compensate. Yawning predictions will carry slightly higher uncertainty than the other two classes; reflect this in your LLM reasoning.

3.3 Preprocessing Pipeline

This is the **exact preprocessing** the backend must apply to every face crop before passing it to the ONNX model.

```python
import torchvision.transforms as T
from PIL import Image

preprocess = T.Compose([
    T.Lambda(lambda img: img.convert("RGB")), # ensure 3-channel
    T.Resize((224, 224)),                      # model input size
    T.ToTensor(),                              # [0,255] HWC ->
        [0,1] CHW
    T.Normalize(
        mean=[0.485, 0.456, 0.406],            # ImageNet mean
        std =[0.229, 0.224, 0.225],            # ImageNet std
    ),
])

face_crop    = Image.open("face_crop.jpg")     # PIL image, any
    initial size
input_tensor = preprocess(face_crop)           # shape: (3, 224,
    224)
```

```
input_batch  = input_tensor.unsqueeze(0)      # shape: (1, 3, 224, 224)
input_numpy  = input_batch.numpy()            # dtype: float32
```

Listing 1: Preprocessing pipeline (must match training exactly).

Property	Value
Shape	`(batch, 3, 224, 224)`
dtype	`float32`
Range	Normalised (≈ -2.1 to $+2.6$)
ONNX input name	`face_image`

Table 4: Input tensor specification for the ONNX model.

4. Model Output Specification

4.1 ONNX Model: `efficientnet.onnx`

The primary deliverable is an EfficientNet-B0 backbone fine-tuned for 3-class fatigue classification and exported to ONNX format.

Property	Value
ONNX output name	`class_logits`
Output shape	`(batch, 3)`
Output dtype	`float32`
Values	Raw unnormalized logits

Table 5: Output tensor specification. Apply softmax before interpreting values.

IMPORTANT The model outputs **raw logits**, not probabilities. You must apply a **softmax** transformation before interpreting the output. Do not compare logit values directly across classes.

4.2 Running Inference

```
import numpy as np
import onnxruntime as ort

sess = ort.InferenceSession(
    "efficientnet.onnx",
    providers=["CUDAExecutionProvider", "CPUExecutionProvider"]
)

# input_numpy shape: (1, 3, 224, 224), dtype float32
logits = sess.run(None, {"face_image": input_numpy})[0]  # (1, 3)

# Apply softmax to get probabilities
probs = np.exp(logits) / np.exp(logits).sum(axis=1, keepdims=True)

prob_microsleep = float(probs[0][0])
```

```python
prob_neutral    = float(probs[0][1])
prob_yawning    = float(probs[0][2])

predicted_class = int(probs[0].argmax())
class_names     = ["microsleep", "neutral", "yawning"]
predicted_label = class_names[predicted_class]

# Combined fatigue probability (classes 0 and 2 both indicate fatigue)
fatigue_prob = prob_microsleep + prob_yawning
```

Listing 2: Loading the ONNX model and obtaining class probabilities.

4.3 Per-Frame JSON Payload

The CV FastAPI endpoint emits the following JSON object for every processed frame. This is what your MCP server will receive.

Listing 3: Per-frame prediction payload emitted by the CV module.

```json
{
  "frame_id": 1042,
  "timestamp_ms": 34706,
  "prediction": {
    "class_id": 0,
    "class_name": "microsleep",
    "fatigue_state": "fatigued",
    "probabilities": {
      "microsleep": 0.781,
      "neutral":    0.061,
      "yawning":    0.158
    },
    "fatigue_probability": 0.939,
    "confidence": 0.781
  },
  "gradcam_available": false
}
```

Field	Type	Description
`frame_id`	int	Sequential frame index from the video stream
`timestamp_ms`	int	Milliseconds since stream start
`class_id`	int	0, 1, or 2
`class_name`	str	`"microsleep"`, `"neutral"`, or `"yawning"`
`fatigue_state`	str	`"fatigued"` if class $\in \{0, 2\}$, else `"alert"`
`probabilities`	object	Softmax probabilities summing to 1.0
`fatigue_probability`	float	$p_{\text{microsleep}} + p_{\text{yawning}}$ — primary signal
`confidence`	float	Probability of the predicted class
`gradcam_available`	bool	True if heatmap bitmap is attached (future)

Table 6: Field definitions for the per-frame prediction payload.

5. Sliding Window & Temporal Context

IMPORTANT The CV model is **frame-level** — it classifies one face crop at a time with no memory of past frames. **All temporal reasoning is the LLM team's responsibility.**

The backend buffers a sliding window of the last N frame predictions and sends the entire window to your MCP server at regular intervals.

Parameter	Suggested Value
Window size	8–16 frames
Stride	4 frames
Send interval	Every 2 seconds

Table 7: Sliding window parameters (confirm with backend team).

Key temporal patterns to detect:

- **Sustained microsleep:** `class_id == 0` for ≥3 consecutive frames → high severity.
- **Repeated yawning:** `class_id == 2` appears ≥2× in the window → moderate warning.
- **Recovery:** transitions from `0` or `2` back to `1` → fatigue improving.
- **Oscillation:** rapid 1 → 0 → 1 → 0 pattern → early-stage fatigue onset.

6. Suggested LLM Input Schema (MCP Payload)

Below is the suggested JSON schema for the MCP message your LLM service receives. Confirm the exact structure with the backend/MCP team.

Listing 4: Suggested MCP payload sent to the LLM service (window of 5 frames shown).

```
{
  "session_id": "driver-abc-2026-09-13",
  "window_frames": [
    { "frame_id": 1038, "class_name": "neutral",
      "fatigue_probability": 0.12, "confidence": 0.88 },
    { "frame_id": 1039, "class_name": "neutral",
      "fatigue_probability": 0.15, "confidence": 0.85 },
    { "frame_id": 1040, "class_name": "yawning",
      "fatigue_probability": 0.73, "confidence": 0.71 },
    { "frame_id": 1041, "class_name": "microsleep",
      "fatigue_probability": 0.94, "confidence": 0.78 },
    { "frame_id": 1042, "class_name": "microsleep",
      "fatigue_probability": 0.91, "confidence": 0.81 }
  ],
  "window_summary": {
    "dominant_class":           "microsleep",
    "mean_fatigue_probability": 0.57,
    "fatigue_frame_count":      3,
    "alert_frame_count":        2,
    "trend":                    "deteriorating"
  }
```

```
}
```

The `trend` field is computed by the backend by comparing the mean `fatigue_probability` of the first and second halves of the window. Possible values: `"stable"`, `"improving"`, `"deteriorating"`.

7. Suggested LLM System Prompt Template

Listing 5: System prompt to use when querying the local LLM (Ollama).

```
You are a real-time driver fatigue monitoring assistant.
You will receive a JSON object containing the last N frame
   predictions
from a computer vision model that detects three states:
  - "microsleep": driver's eyes are closing involuntarily
  - "neutral"   : driver is alert with eyes open
  - "yawning"   : driver is yawning

Your task is to:
1. Analyse the temporal pattern of states in the window.
2. Assess severity on a scale: NONE / LOW / MODERATE / HIGH /
   CRITICAL.
3. Explain what specific visual evidence (class sequence) led to
   this assessment.
4. Output a brief plain-English alert suitable for display to
   the driver.

Always respond in valid JSON with this exact structure:
{
  "severity": "<NONE|LOW|MODERATE|HIGH|CRITICAL>",
  "reasoning": "<2-3 sentences explaining the evidence>",
  "alert_message": "<Short driver-facing message, max 15 words>",
  "recommended_action": "<REST_BREAK | CAUTION | NONE>"
}
```

Example LLM output for the payload shown in Section 6:

Listing 6: Expected LLM JSON response for the example window.

```
{
  "severity": "HIGH",
  "reasoning": "The window shows a clear deteriorating trend: the
    driver began yawning in frame 1040 and transitioned into two
    consecutive microsleep episodes in frames 1041-1042. Mean
       fatigue
    probability reached 0.57 across the window and 0.93 in the
       final
    two frames.",
  "alert_message": "Warning: eyes closing detected. Please rest
     soon.",
  "recommended_action": "REST_BREAK"
}
```

8. Severity Threshold Guidelines

Use these thresholds when computing the severity rating from `fatigue_probability` and temporal pattern counts. These are starting guidelines and should be tuned once real inference data is available.

Severity	Condition
NONE	All frames `neutral`; mean `fatigue_prob` < 0.15.
LOW	1–2 yawning frames in window; mean `fatigue_prob` < 0.35.
MODERATE	≥ 2 yawning frames **or** 1 microsleep frame; mean `fatigue_prob` $\in [0.35, 0.60)$.
HIGH	1–2 microsleep frames **or** sustained yawning; mean `fatigue_prob` $\in [0.60, 0.80)$.
CRITICAL	≥ 3 microsleep frames **or** `trend == "deteriorating"` with mean `fatigue_prob` > 0.80.

Table 8: Severity thresholds for the LLM alerting layer.

9. Files Delivered by the CV Team

File	Description
`efficientnet.onnx`	Primary model. Load with `onnxruntime`. Input: face image $(1, 3, 224, 224)$ float32. Output: logits $(1, 3)$.
`efficientnet_interface.txt`	Auto-generated interface spec: input/output shapes, normalization parameters, class mapping.
`custom_cnn.onnx`	Lightweight baseline model. Optional; useful for fallback or benchmarking. Input: $(1, 1, 96, 96)$ grayscale float32.
`evaluation_report.txt`	Per-class F1, ROC-AUC, and confusion matrix on the held-out test set.
`EfficientNet-B0_cm.png`	Confusion matrix plot for the primary model.
`roc_comparison.png`	OVR ROC curves for both models across all three classes.
`efficientnet_gradcam.png`	Sample Grad-CAM attention heatmaps showing which facial regions influenced each prediction.

Table 9: Files the CV team will hand off upon training completion.

10. Open Questions for Team Alignment

Before starting implementation, please confirm the following with the backend and MCP teams:

1. **Window delivery:** Does the Java gateway buffer frames and push the window to your MCP server, or do you pull from a stream?

2. **Inference frame rate:** At what FPS does the CV module run? (Target: 12–24 FPS on a mid-tier GPU with ONNX Runtime.)

3. **Latency budget:** How many milliseconds does the LLM have to respond before the alert is considered stale?

4. **Alert UI rendering:** Does the React UI render the `alert_message` string directly, or does it parse the full severity JSON to display a styled component?

5. **Grad-CAM heatmaps:** Do you want per-frame heatmap data attached to the payload? This can be offered as a separate async endpoint to avoid adding latency to the main inference path.

For CV model-specific questions, contact the CV team.
This document will be updated when trained model checkpoints are available.